

A Brain-Friendly Guide

Head First C



Discover the secrets
of the C coding gurus



Learn how make can
change your life



Avoid
embarrassing
pointer
mistakes

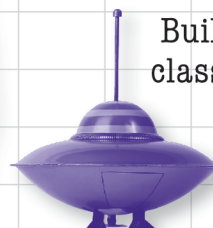
See how variadic
functions helped
Sue be more
flexible



Fool around
in the C
Standard
Library



Build a retro
classic arcade
game

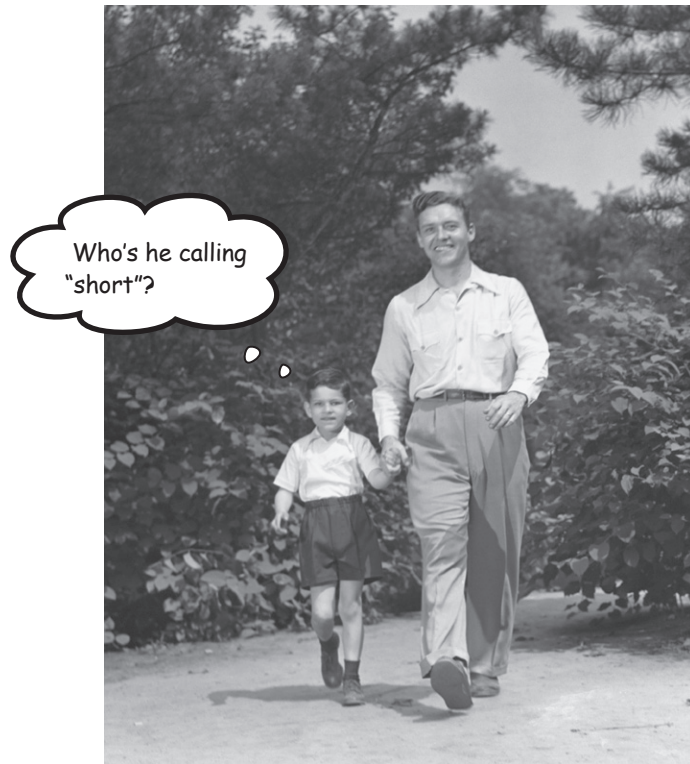


O'REILLY®

David Griffiths &
Dawn Griffiths

4 using multiple source files

***Break it down,
build it up***



If you create a big program, you don't want a big source file.

Can you imagine how difficult and time-consuming a single source file for an enterprise-level program would be to maintain? In this chapter, you'll learn how C allows you to break your source code into **small, manageable chunks** and then rebuild them into **one huge program**. Along the way, you'll learn a bit more about **data type subtleties** and get to meet your new best friend: **make**.

Your quick guide to data types

char

Each character is stored in the computer's memory as a character code. And that's just a number. So when the computer sees A, to the computer it's the same as seeing the literal number 65.

int

↖ 65 is the ASCII
code for A.

If you need to store a whole number, you can generally just use an `int`. The exact maximum size of an `int` can vary, but it's guaranteed to be at least 16 bits. In general, an `int` can store numbers up to a few million.

short

But sometimes you want to save a little memory. Why use an `int` if you just want to store numbers up to a few hundreds or thousands? That's what a `short` is for. A `short` number usually takes up about half the space of an `int`.

long

Yes, but what if you want to store a **really large count**? That's what the `long` data type was invented for. On some machines, the `long` data type takes up *twice* the memory of an `int`, and it can hold numbers up in the **billions**. But because most computers can deal with really large `ints`, on a lot of machines, the `long` data type is *exactly the same size* as an `int`. The maximum size of a `long` is guaranteed to be at least 32 bits.

float

`float` is the basic data type for storing floating-point numbers. For most everyday floating-point numbers—like the amount of fluid in your orange mocha frappuccino—you can use a `float`.

double

Yes, but what if you want to get really **precise**? If you want to perform calculations that are accurate to a large number of **decimal places**, then you might want to use a `double`. A `double` takes up twice the memory of a `float`, and it uses that extra space to store numbers that are *larger and more precise*.

Don't put something big into something small

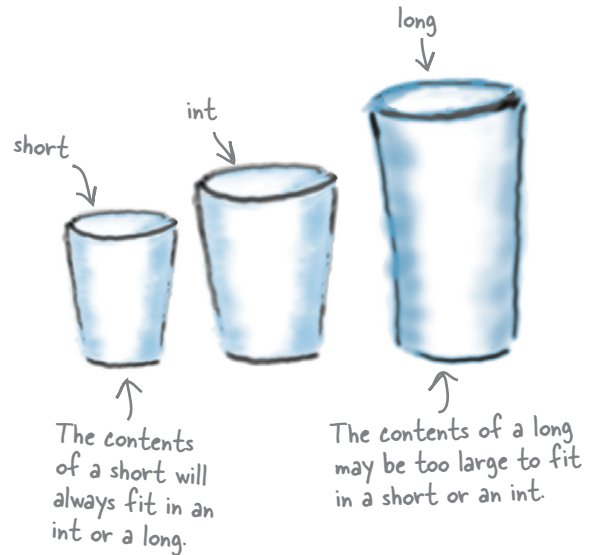
When you're passing around values, you need to be careful that the type of the value matches the type of the variable you are going to store it in.

Different data types use different amounts of memory. So you need to be careful that you don't try to store a value that's too large for the amount of space allocated to a variable. `short` variables take up less memory than `ints`, and `ints` take up less memory than `longs`.

Now there's no problem storing a `short` value inside an `int` or a `long` variable. There is plenty of space in memory, and your code will work correctly:

```
short x = 15;
int y = x;
printf("The value of y = %i\n", y);
```

This will say that $y = 15$.



The problems start to happen if you go the other way around—if, say, you try to store an `int` value into a `short`.

```
int x = 100000;
short y = x;
print("The value of y = %hi\n", y);
```

`%hi` is the proper code to format a short value.

Sometimes, the compiler will be able to spot that you're trying to store a really big value into a small variable, and then give you a warning. But a lot of the time the compiler won't be smart enough for that, and it will compile the code without complaining. In that case, when you try to run the code, the computer won't be able to store a number 100,000 into a `short` variable. The computer will fit in as many 1s and 0s as it can, but the number that ends up stored inside the `y` variable will be *very different* from the one you sent it:

The value of `y` = -31072



Geek Bits

So why did putting a large number into a `short` go negative? Numbers are stored in binary. This is what 100,000 looks like in binary:

```
x <- 0001 1000 0110 1010 0000
```

But when the computer tried to store that value into a `short`, it only allowed the value a couple of bytes of storage. The program stored just the *righthand side* of the number:

```
y <- 1000 0110 1010 0000
```

Signed values in binary beginning with a 1 in highest bit are treated as negative numbers. And this shortened value is equal to this in decimal:

```
-31072
```


Use casting to put floats into whole numbers

What do you think this piece of code will display?

```
int x = 7;
int y = 2;
float z = x / y;
printf("z = %f\n", z);
```

The answer? **3.0000**. Why is that? Well, `x` and `y` are both integers, and if you divide integers you always get a rounded-off whole number—in this case, **3**.

What do you do if you want to perform calculations on whole numbers and you want to get floating-point results? You could store the whole numbers into `float` variables first, but that's a little wordy. Instead, you can use a **cast** to convert the numbers on the fly:

```
int x = 7;
int y = 2;
float z = (float)x / (float)y;
printf("z = %f\n", z);
```

The **(float)** will *cast* an integer value into a `float` value. The calculation will then work just as if you were using floating-point values the entire time. In fact, if the compiler sees you are adding, subtracting, multiplying, or dividing a floating-point value with a whole number, it will automatically cast the numbers for you. That means you can cut down the number of explicit casts in your code:

```
float z = (float)x / y; ← The compiler will automatically
                        cast y to a float.
```



You can put some other keywords before data types to change the way that the numbers are interpreted:

unsigned

The number will always be positive. Because it doesn't need to worry about recording negative numbers, `unsigned` numbers can store larger numbers since there's now one more bit to work with. So an `unsigned int` stores numbers from 0 to a maximum value that is about twice as large as the maximum number that can be stored inside an `int`. There's also a signed keyword, but you almost never see it, because all data types are signed by default.

```
unsigned char c;
```

↖ This will probably store numbers from 0 to 255.

long

That's right, you can prefix a data type with the word `long` and make it longer. So a `long int` is a longer version of an `int`, which means it can store a larger range of numbers. And a `long long` is longer than a `long`. You can also use `long` with floating-point numbers.

↗ `long double d;`
A really REALLY precise number.

↖ `long long` is C99 and C11 only.



Exercise

There's a new program helping the waiters bus tables at the Head First Diner. The code automatically totals a bill and adds sales tax to each item. See if you can figure out what needs to go in each of the blanks.

Note: there are several data types that could be used for this program, but which would you use for the kind of figures you'd expect?

```
#include <stdio.h>

.....total = 0.0;
.....count = 0;
.....tax_percent = 6;

.....add_with_tax(float f);
{
    .....tax_rate = 1 + tax_percent / 100 ..... ;
    total = total + (f * tax_rate);
    count = count + 1;
    return total;
}

int main()
{
    .....val;
    printf("Price of item: ");
    while (scanf("%f", &val) == 1) {
        printf("Total so far: %.2f\n", add_with_tax(val));
        printf("Price of item: ");
    }
    printf("\nFinal total: %.2f\n", total);
    printf("Number of items: %hi\n", count);
    return 0;
}
```

%.2f formats a floating-point number to two decimal places.

%hi is used to format shorts.

Data Type Sizes Up Close

Data types are different sizes on different platforms. But how do you find out how big an `int` is, or how many bytes a `double` takes up? Fortunately, the C Standard Library has a couple of headers with the details. This program will tell you about the sizes of `ints` and `floats`:

```
#include <stdio.h>
#include <limits.h> ← This contains the values for the integer types like int and char.
#include <float.h> ← This contains the values for floats and doubles.
```

```
int main()
{
    printf("The value of INT_MAX is %i\n", INT_MAX);
    printf("The value of INT_MIN is %i\n", INT_MIN);
    printf("An int takes %z bytes\n", sizeof(int));

    printf("The value of FLT_MAX is %f\n", FLT_MAX);
    printf("The value of FLT_MIN is %.50f\n", FLT_MIN);
    printf("A float takes %z bytes\n", sizeof(float));

    return 0;
}
```

This is the highest value.

✓ This is the lowest value.

sizeof returns the number of bytes a data type occupies.

When you compile and run this code, you will see something like this:

[illegible]

The values you see on your particular machine will probably be different.

What if you want to know the details for chars or doubles? Or longs? No problem. Just replace INT and FLT with CHAR (chars), DBL (doubles), SHRT (shorts), or LNG (longs).

there are no Dumb Questions

Q: Why are data types different on different operating systems? Wouldn't it be less confusing to make them all the same?

A: C uses different data types on different operating systems and processors because that allows it to make the most out of the hardware.

Q: In what way?

A: When C was first created, most machines were 8-bit. Now, most machines are 32- or 64-bit. Because C doesn't specify the exact size of its data types, it's been able to adapt over time. And as newer machines are created, C will be able to make the most of them as well.

Q: What do 8-bit and 64-bit actually mean?

A: Technically, the bit size of a computer can refer to several things, such as the size of its CPU instructions or the amount of data the CPU can read from memory. The bit size is really the favored size of numbers that the computer can deal with.

Q: So what does that have to do with the size of `ints` and `doubles`?

A: If a computer is optimized best to work with 32-bit numbers, it makes sense if the basic data type—the `int`—is set at 32 bits.

Q: I understand how whole numbers like `ints` work, but how are `floats` and `doubles` stored? How does the computer represent a number with a decimal point?

A: It's complicated. Most computers used a standard published by the IEEE (<http://tinyurl.com/6defkv6>).

Q: Do I really need to understand how floating-point numbers work?

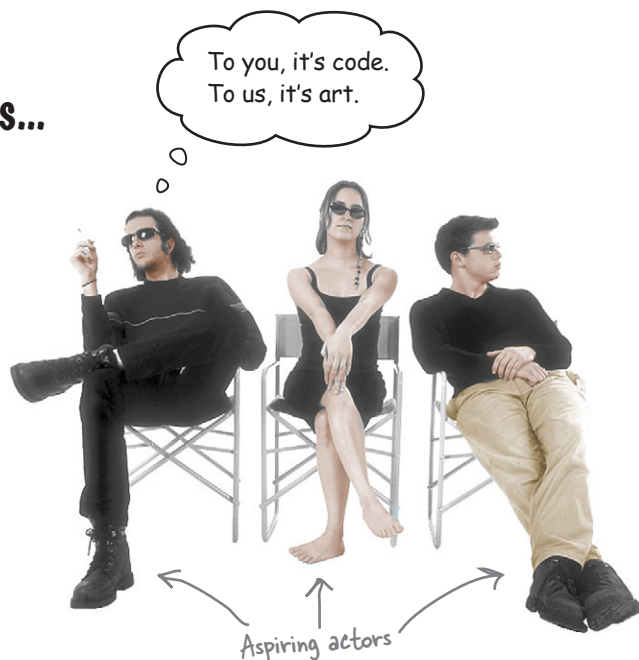
A: No. The vast majority of developers use `floats` and `doubles` without worrying about the details.

Oh no...it's the out-of-work actors...

Some people were never really cut out to be programmers. It seems that some aspiring actors are filling in their time *between roles* and making a little extra cash by cutting code, and they've decided to spend some time freshening up the code in the bill-totalling program.

By the time they rejiggered the code, the actors were much happier about the way everything looked...but there's just a tiny problem.

The code doesn't compile anymore.



Let's see what's happened to the code

This is what the actors did to the code. You can see they really just did a couple of things.

```
#include <stdio.h>

float total = 0.0;
short count = 0;
/* This is 6%. Which is a lot less than my agent takes...*/
short tax_percent = 6;

int main()
{
    /* Hey - I was up for a movie with Val Kilmer */
    float val;
    printf("Price of item: ");
    while (scanf("%f", &val) == 1) {
        printf("Total so far: %.2f\n", add_with_tax(val));
        printf("Price of item: ");
    }
    printf("\nFinal total: %.2f\n", total);
    printf("Number of items: %hi\n", count);
    return 0;
}

float add_with_tax(float f)
{
    float tax_rate = 1 + tax_percent / 100.0;
    /* And what about the tip? Voice lessons ain't free */
    total = total + (f * tax_rate);
    count = count + 1;
    return total;
}
```

The code has had some comments added, and they also **changed the order of the functions**. They made no other changes.

So there really shouldn't be a problem. The code should be good to go, right? Well, everything was great, right up until the point that they **compiled the code...**



Test Drive

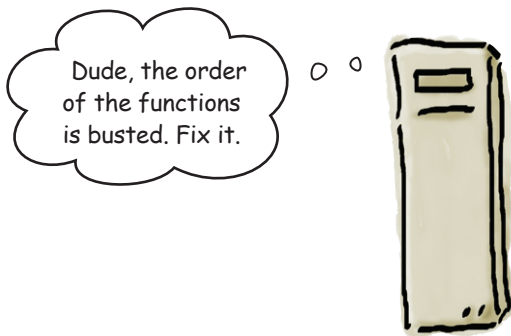
If you open up the console and try to compile the program, this happens:

```
File Edit Window Help StickToActing
> gcc totaller.c -o totaller && ./totaller
totaller.c: In function "main":
totaller.c:14: warning: format "%.2f" expects type
"double", but argument 2 has type "int"
totaller.c: At top level:
totaller.c:23: error: conflicting types for "add_with_tax"
totaller.c:14: error: previous implicit declaration of
"add_with_tax" was here
```

Bummer.

That's not good. What does `error: conflicting types for 'add_with_tax'` mean? What is a *previous implicit declaration*? And why does it think the line that prints out the current total is now an `int`? Didn't we design that to be floating point?

The compiler will ignore the changes made to the comments, so that shouldn't make any difference. That means the problem must be caused by **changing the order of the functions**. But if the order is the problem, why doesn't the compiler just return a message saying something like:



Seriously, why doesn't the compiler give us a little help here?

To understand exactly what's happening here, you need to get inside the head of the compiler for a while and look at things from its point of view. You'll see that what's happening is that the compiler is actually trying to be a little *too helpful*.

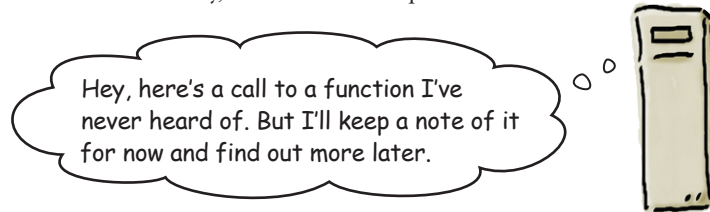
Compilers don't like surprises

So what happens when the compiler sees this line of code?

```
printf("Total so far: %.2f\n", add_with_tax(val));
```

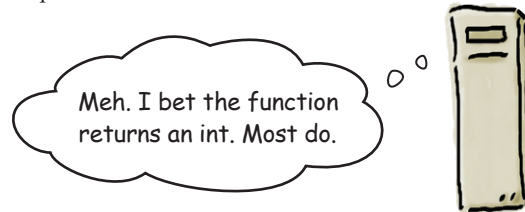
1 The compiler sees a call to a function it doesn't recognize.

Rather than complain about it, the compiler figures that it will find out more about the function later in the source file. The compiler simply remembers to look out for the function later on in the file. Unfortunately, this is where the problem lies...



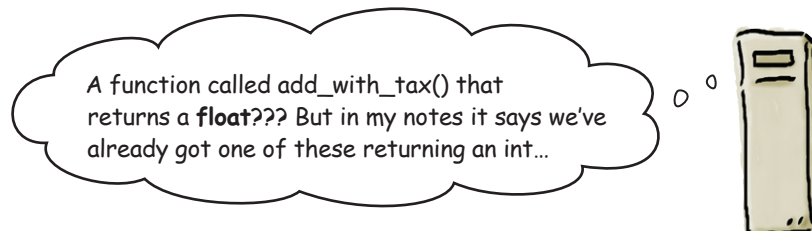
2 The compiler needs to know what data type the function will return.

Of course, the compiler can't know what the function will return just yet, so it makes an **assumption**. The compiler assumes it will return an `int`.



3 When it reaches the code for the actual function, it returns a "conflicting types for 'add_with_tax'" error.

This is because the compiler thinks it has two functions with the same name. One function is the real one in the file. The other is the one that the compiler assumed would return an `int`.



The computer makes an assumption that the function returns an `int`, when in reality it returns a `float`. If you were designing the C language, how would you fix the problem?

Hello? I really don't **care** how the C language solves the problem. Just put the functions in the **correct freaking order**!

You could just put the functions back in the correct order and define the function before you call it in main().

Changing the order of the functions means that you can avoid the compiler ever making any dangerous assumptions about the return types of unknown functions. But if you force yourself to always define functions in a specific order, there are a couple of consequences.



Fixing function order is a pain

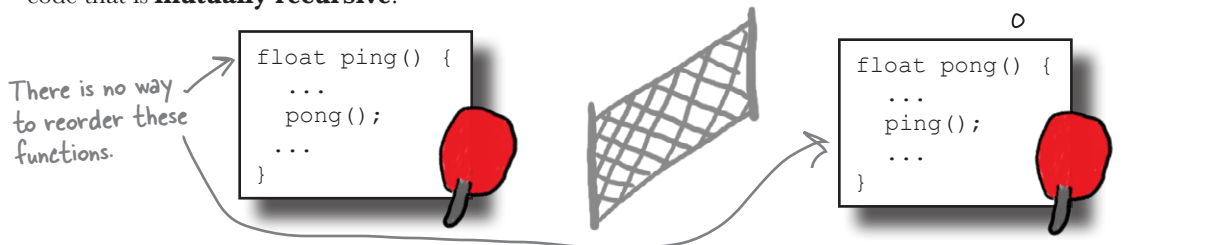
Say you've added a cool new function to your code that everyone thinks is fantastic:

```
int do_whatever(){...}
float do_something_fantastic(int awesome_level) {...}
int do_stuff() {
    do_something_fantastic(11);
}
```

What happens if you *then* decide your program will be even *better* if you add a call to the `do_something_fantastic()` function in the existing `do_whatever()` code? You will have to **move the function** earlier in the file. Most coders want to spend their time improving what their code can do. It would be better if you didn't have to shuffle the order of the code just to keep the compiler happy.

In some situations, there is no correct order

OK, so this situation is kind of rare, but occasionally you might write some code that is **mutually recursive**:



If you have two functions that call *each other*, then **one of them will always be called in the file before it's defined**.

For both of those reasons, it's really useful to be able to define functions in whatever order is easiest at the time. But how?

Split the declaration from the definition

Remember how the compiler made a note to itself about the function it was expecting to find later in the file? You can avoid the compiler making assumptions by **explicitly telling it what functions it should expect**. When you tell the compiler about a function, it's called a **function declaration**:

The declaration tells the compiler what return value to expect. → `float add_with_tax();` ← A declaration has no body code.
 ← It just ends with a ; (semicolon).

The declaration is just a function **signature**: a record of what the function will be called, what kind of parameters it will accept, and **what type of data it will return**.

Once you've declared a function, the compiler won't need to make any assumptions, so it won't matter if you define the function after you call it.

So if you have a whole bunch of functions in your code and you don't want to worry about their order in the file, you can put a list of function declarations at the start of your C program code:

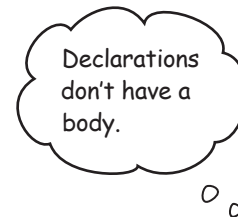
```
float do_something_fantastic();
double awesomeness_2_dot_0();
int stinky_pete();
char make_maguerita(int count);
```

But even better than that, C allows you to take that whole set of declarations *out of your code* and put them in a **header file**. You've already used header files to include code from the C Standard Library:

```
#include <stdio.h>
```

← This line will include the contents of the header file called `stdio.h`.

Let's go see how you can create your own header files.



Creating your first header file

To create a header, you just need to do **two things**:

1 Create a new file with a `.h` extension.

If you are writing a program called `total`, then create a file called `total.h` and write your declarations inside it:

```
float add_with_tax(float f);
```



total.h

You won't need to include the `main()` function in the header file, because nothing else will need to call it.

2 Include your header file in your main program.

At the top of your program, you should add an extra `include` line:

Add this include to your other include lines.

```
#include <stdio.h>
#include "total.h"
...
```



total.c

When you write the name of the header file, make sure you surround it with double quotes rather than angle brackets. Why the difference? When the compiler sees an `include` line with angle brackets, it assumes it will find the header file somewhere off in the directories where the library code lives. But **your** header file is in the same directory as your `.c` file. By wrapping the header filename in quotes, you are telling the compiler to look for a local file.

When the compiler reads the `#include` in the code, it will read the contents of the header file, just as if it had been typed into the code.

Separating the declarations into a separate header file keeps your main code a little shorter, and it has another *big advantage* that you'll find out about in a few pages.

For now, let's see if the header file fixed the mess.

← Local header files can also include directory names, but you will normally put them in the same directory as the C file.

**#include is a
preprocessor
instruction.**



Test Drive

Now when you compile the code, this happens:

No error messages this time.

```
File Edit Window Help UseHeaders
> gcc totaller.c -o totaller
```

The compiler reads the function declarations from the header file, which means it doesn't have to make any guesses about the return type of the function. The order of the functions doesn't matter.

Just to check that everything is OK, you can run the generated program to see if it works the same as before.

```
File Edit Window Help UseHeaders
> ./totaller
Price of item: 1.23
Total so far: 1.30
Price of item: 4.57
Total so far: 6.15
Price of item: 11.92
Total so far: 18.78
Price of item: ^D
Final total: 18.78
Number of items: 3
```

Press Ctrl-D here to stop the program from asking for more prices.

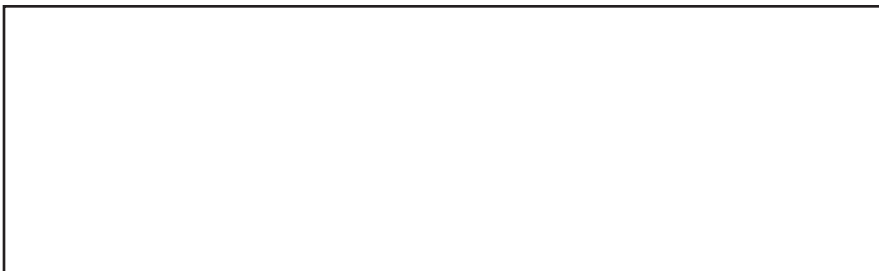


BE the Compiler

Look at the program below. Part of the program is missing. Your job is to play like you're the compiler and say what you would do if each of the candidate code fragments on the right were slotted into the missing space.

Candidate code goes here.

```
#include <stdio.h>
```



```
printf("A day on Mercury is %f hours\n", day);  
return 0;
```

```
}
```

```
float mercury_day_in_earth_days()
```

```
{
```

```
    return 58.65;
```

```
}
```

```
int hours_in_an_earth_day()
```

```
{
```

```
    return 24;
```

```
}
```

Here are the code fragments. →

```
float mercury_day_in_earth_days();
int hours_in_an_earth_day();

int main()
{
    float length_of_day = mercury_day_in_earth_days();
    int hours = hours_in_an_earth_day();
    float day = length_of_day * hours;
```

☐

You can compile the code.

☐

You should display a warning.

☐

The program will work.

↙ Mark the boxes that you think are correct.

```
float mercury_day_in_earth_days();

int main()
{
    float length_of_day = mercury_day_in_earth_days();
    int hours = hours_in_an_earth_day();
    float day = length_of_day * hours;
```

☐

You can compile the code.

☐

You should display a warning.

☐

The program will work.

```
int main()
{
    float length_of_day = mercury_day_in_earth_days();
    int hours = hours_in_an_earth_day();
    float day = length_of_day * hours;
```

☐

You can compile the code.

☐

You should display a warning.

☐

The program will work.

```
float mercury_day_in_earth_days();
int hours_in_an_earth_day();

int main()
{
    int length_of_day = mercury_day_in_earth_days();
    int hours = hours_in_an_earth_day();
    float day = length_of_day * hours;
```

☐

You can compile the code.

☐

You should display a warning.

☐

The program will work.

there are no
Dumb Questions

Q: So I don't need to have declarations for `int` functions?

A: Not necessarily, unless you are sharing code. You'll see more about this soon.

Q: I'm confused. You talk about the compiler *preprocessing*? Why does the compiler do that?

A: Strictly speaking, the compiler just does the compilation step: it converts the C source code into assembly code. But in a looser sense, all of the stages that convert the C source code into the final executable are normally called *compilation*, and the `gcc` tool allows you to control those stages. The `gcc` tool does preprocessing and compilation.

Q: What is the preprocessor?

A: Preprocessing is the first stage in converting the raw C source code into a working executable. Preprocessing creates a modified version of the source just before the *proper* compilation begins. In your code, the preprocessing step read the contents of the header file into the main file.

Q: Does the preprocessor create an actual file?

A: No, compilers normally just use pipes for sending the stuff through the phases of the compiler to make things more efficient.

Q: Why do some headers have quotes and others have angle brackets?

A: Strictly speaking, it depends on the way your compiler works. Usually quotes mean to simply look for a file using a relative path. So if you just include the name of a file, without including a directory name, the compiler will look in the current directory. If angle brackets are used, it will search for the file along a path of directories.

Q: What directories will the compiler search when it is looking for header files?

A: The `gcc` compiler knows where the standard headers are stored. On a Unix-style operating system, the header files are normally in places like `/usr/local/include`, `/usr/include`, and a few others.

Q: So that's how it works for standard headers like `stdio.h`?

A: Yes. You can read through the `stdio.h` file on a Unix-style machine in `/usr/include/stdio.h`. If you have the MinGW compiler on Windows, it will probably be in `C:\MinGW\include\stdio.h`.

Q: Can I create my own libraries?

A: Yes; you'll learn how to do that later in the book.



BULLET POINTS

- If the compiler finds a call to a function it hasn't heard of, it will assume the function returns an `int`.
- So if you try to call a function before you define it, there can be problems.
- Function declarations tell the compiler what your functions will look like before you define them.
- If function declarations appear at the top of your source code, the compiler won't get confused about return types.
- Function declarations are often put into header files.
- You can tell the compiler to read the contents of a header file using `#include`.
- The compiler will treat `included` code the same as code that is typed into the source file.



This Table's Reserved...

C is a very small language. Here is the entire set of reserved words (in no useful order).

Every C program you ever see will break into just these words and a few symbols. If you use these for names, the compiler will be very, very upset.

<code>auto</code>	<code>if</code>	<code>break</code>
<code>int</code>	<code>case</code>	<code>long</code>
<code>char</code>	<code>register</code>	<code>continue</code>
<code>return</code>	<code>default</code>	<code>short</code>
<code>do</code>	<code>sizeof</code>	<code>double</code>
<code>static</code>	<code>else</code>	<code>struct</code>
<code>entry</code>	<code>switch</code>	<code>extern</code>
<code>typedef</code>	<code>float</code>	<code>union</code>
<code>for</code>	<code>unsigned</code>	<code>goto</code>
<code>while</code>	<code>enum</code>	<code>void</code>
<code>const</code>	<code>signed</code>	<code>volatile</code>

If you have common features...

Chances are, when you begin to write several programs in C, you will find that there are some functions and features that you will want to reuse from other programs. For example, look at the specs of the two programs on the right.

XOR encryption is a very simple way of disguising a piece of text by XOR-ing each character with some value. It's not very secure, but it's very easy to do. And the same code that can encrypt text can also be used to decrypt it. Here's the code to encrypt some text:

void means don't return anything.

```
void encrypt(char *message)
{
    char c;
    while (*message) {
        *message = *message ^ 31;
        message++;
    }
}
```

Loop through the array and update each character with an encrypted version.

Pass a pointer to an array into the function.

This means you'll XOR each character with the number 31.

Doing math with a character? You can because char is a numeric data type.

file_hider

Read the contents of a file and create an encrypted version using XOR encryption.

message_hider

Read a series of strings from the standard input and display an encrypted version on the standard output using XOR encryption.

...it's good to share code

Clearly, both of those programs are going to need to use the same `encrypt()` function. So you could just copy the code from one program to the other, right? That's not so bad if there's just a small amount of code to copy, but what if there's a really large amount of code? Or what if the way the `encrypt()` function works needs to change in the future? If there are two copies of the `encrypt()` function, you will have to change it in more than one place.

For your code to scale properly, you really need to find some way to reuse common pieces of code—some way of taking a set of functions and making them available in a bunch of different programs.

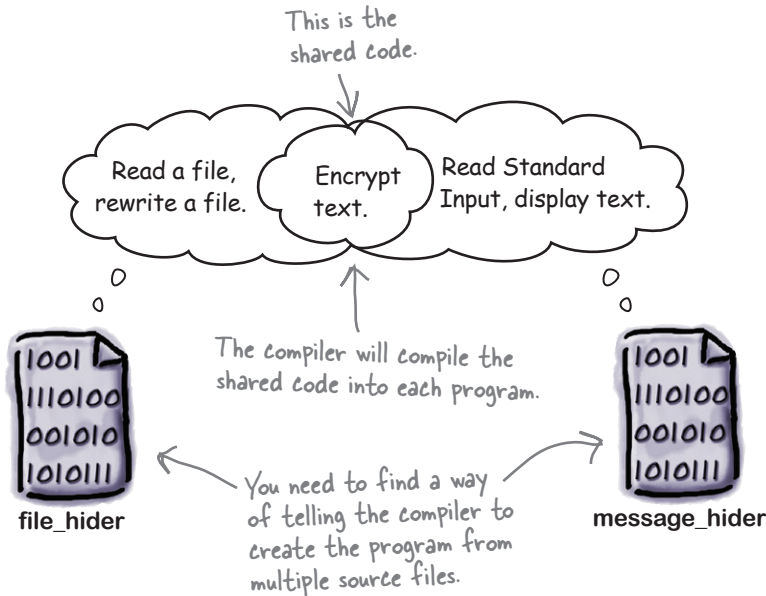
How would you do that?



Imagine you have a set of functions that you want to share between programs. If you had created the C programming language, how would you allow code to be shared?

You can split the code into separate files

If you have a set of code that you want to share among several files, it makes a lot of sense to put that shared code into a separate `.c` file. If the compiler can somehow include the shared code when it's compiling the program, you can use the same code in multiple applications at once. So if you ever need to change the shared code, you only have to do it in one place.



If you want to use a separate `.c` file for the shared code, that gives us a *problem*. So far, you have only created programs from single `.c` source files. So if you had a C program called `blitz_hack`, you would have created it from a single source code file called `blitz_hack.c`.

But now you want some way to give the compiler a **set of source code files** and say, "Go make a program from those." How do you do that? What syntax do you use with the `gcc` compiler? And more importantly, what does it *mean* for a compiler to create a single executable program from several files? How would it work? How would it stitch them together?

To understand how the C compiler can create a single program from multiple files, let's take a look at how compilation works...

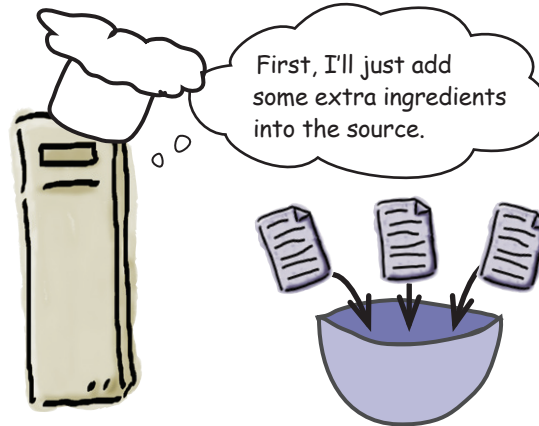
Compilation behind the scenes

To understand how a compiler can compile several source files into a single program, you'll need to pull back the curtain and see how compilation really works.

1 Preprocessing: fix the source.

The first thing the compiler needs to do is fix the source. It needs to add in any extra header files it's been told about using the `#include` **directive**. It might also need to expand or skip over some sections of the program. Once it's done, the source code will be ready for the actual compilation.

It can do this with commands like `#define` and `#ifdef`. You'll see how to use them later in the book.



Hmmm...so I need to compile the source files into a program? Let's see what I can cook up...

"directive" is just a fancy word for "command."

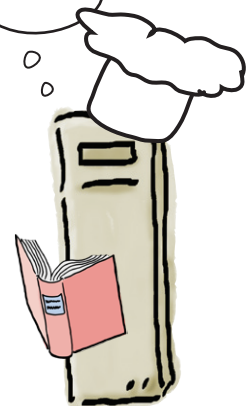
2 Compilation: translate into assembly.

The C programming language probably seems pretty low level, but the truth is it's *not low level enough* for the computer to understand. The computer only really understands very low-level **machine code** instructions, and the first step to generate machine code is to convert the C source code into **assembly language symbols** like this:

```
movq -24(%rbp), %rax
movzbl(%rax), %eax
movl %eax, %edx
```

Looks pretty obscure? Assembly language describes the individual instructions the central processor will have to follow when running the program. The C compiler has a whole set of recipes for each of the different parts of the C language. These recipes will tell the compiler how to convert an `if` statement or a function call into a sequence of assembly language instructions. But even assembly isn't low level enough for the computer. That's why it needs...

So for this "if" statement I need to begin by adding onto the stack...



3

Assembly: generate the object code.

The compiler will need to *assemble* the symbol codes into *machine* or **object code**. This is the actual binary code that will be executed by the circuits inside the CPU.

This is a really
dirty joke in
machine code.

10010101 00100101 11010101 01011100

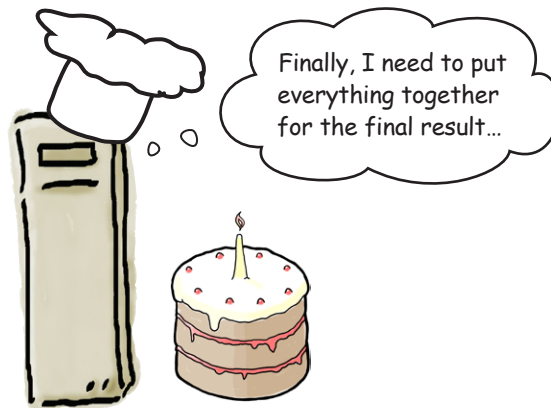
So are you all done? After all, you've taken the original C source code and converted it into the 1s and 0s that the computer's circuits need. But no, there's still one more step. If you give the computer several files to compile for a program, the compiler will generate a piece of object code for each source file. But in order for these separate object files to form a single executable program, one more thing has to occur...



4

Linking: put it all together.

Once you have all of the separate pieces of object code, you need to fit them together like jigsaw pieces to form the **executable program**. The compiler will connect the code in one piece of object code that calls a function in another piece of object code. Linking will also make sure that the program is able to call library code properly. Finally, the program will be written out into the executable program file using a format that is supported by the operating system. The file format is important, because it will allow the operating system to load the program into memory and make it run.



So how do you actually tell gcc that we want to make one executable program from several separate source files?

The shared code needs its own header file

If you are going to share the *encrypt.c* code between programs, you need some way to tell those programs about the encrypt code. You do that with a header file.

You'll include the header inside *encrypt.c*.

```
void encrypt(char *message);
```

encrypt.h

```
#include "encrypt.h"
```

```
void encrypt(char *message)
{
    char c;
    while (*message) {
        *message = *message ^ 31;
        message++;
    }
}
```

encrypt.c

Include *encrypt.h* in your program

You're not using a header file here to be able to reorder the functions. You're using it to **tell other programs about the *encrypt()* function**:

```
#include <stdio.h>
#include "encrypt.h"

int main()
{
    char msg[80];
    while (fgets(msg, 80, stdin)) {
        encrypt(msg);
        printf("%s", msg);
    }
}
```

message_hider.c

You'll include *encrypt.h* so that the program has the declaration of the *encrypt()* function.

Having *encrypt.h* inside the main program will mean the compiler will know enough about the *encrypt()* function to compile the code. At the linking stage, the compiler will be able to connect the call to *encrypt(msg)* in *message_hider.c* to the actual *encrypt()* function in *encrypt.h*.

Finally, to compile everything together you just need to pass the source files to *gcc*:

```
gcc message_hider.c encrypt.c -o message_hider
```

Sharing variables

You've seen how to share functions between different files. But what if you want to share variables? Source code files normally contain their own separate variables to prevent a variable in one file affecting a variable in another file with the same name. But if you genuinely want to share variables, you should declare them in your header file and prefix them with the keyword **extern**:

```
extern int passcode;
```



Test Drive

Let's see what happens when you compile the `message_hider` program:

When you run the program, you can enter text and see the encrypted version.

You can even pass it the contents of the `encrypt.h` file to encrypt it.

The `message_hider` program is using the `encrypt()` function from `encrypt.c`.

```
File Edit Window Help Shhh...
> gcc message_hider.c encrypt.c -o message_hider
> ./message_hider
I am a secret message
V?~r?~?1z|mzk?rz11~xz
> ./message_hider < encrypt.h
ipv{?zq|mfoK7|w~m5?rz11~xz6$
>
```

You need to compile the code with both source files.

The program works. Now that you have the `encrypt()` function in a separate file, you can use it in any program you like. If you ever change the `encrypt()` function to be something a little more secure, you will need to amend only the `encrypt.c` file.



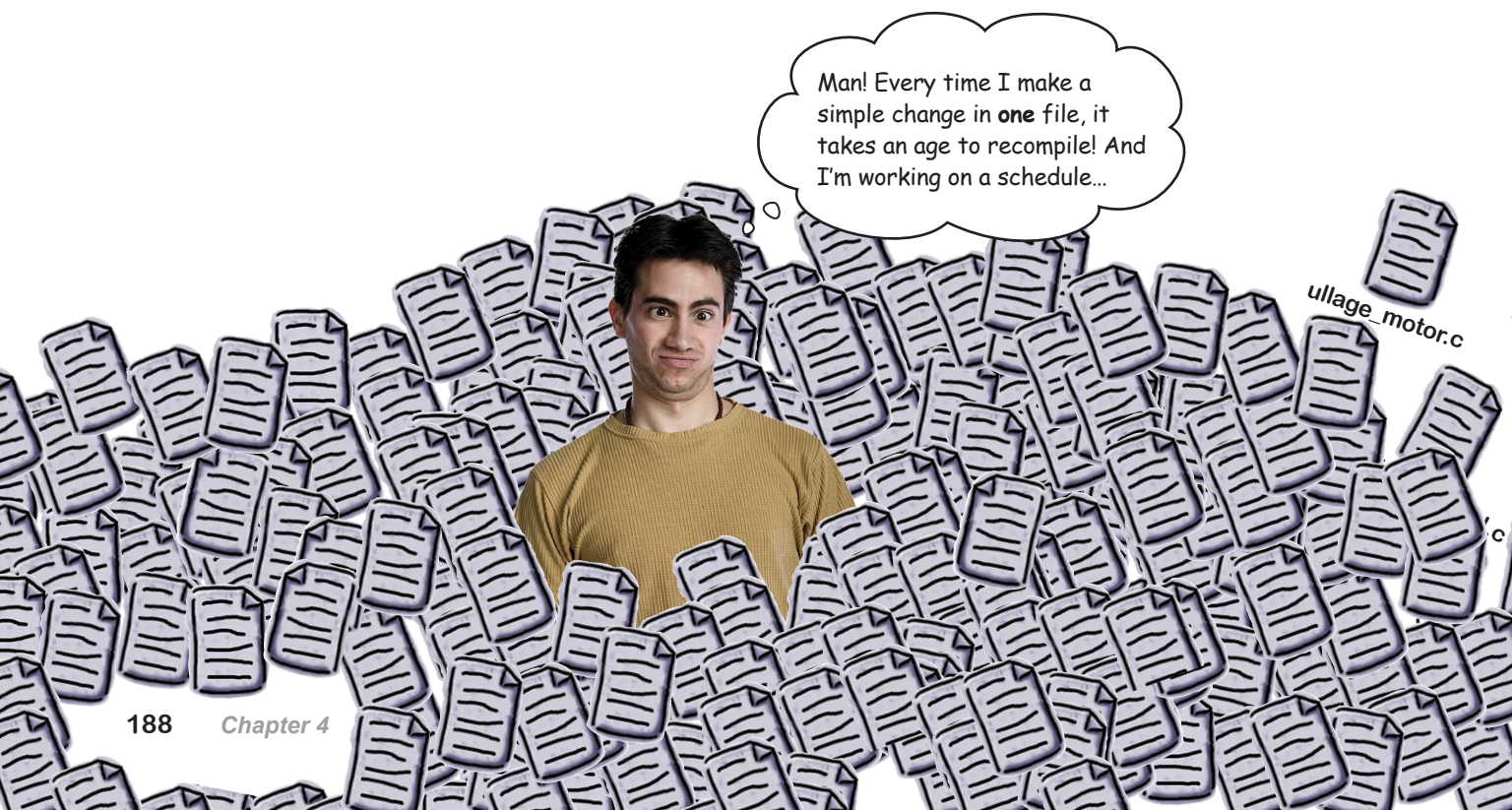
BULLET POINTS

- You can share code by putting it into a separate C file.
- You need to put the function declarations in a separate `.h` header file.
- Include the header file in every C file that needs to use the shared code.
- List all of the C files needed in the compiler command.



Go Off Piste

Write your own program using the `encrypt()` function. Remember, you can call the same function to decrypt text.

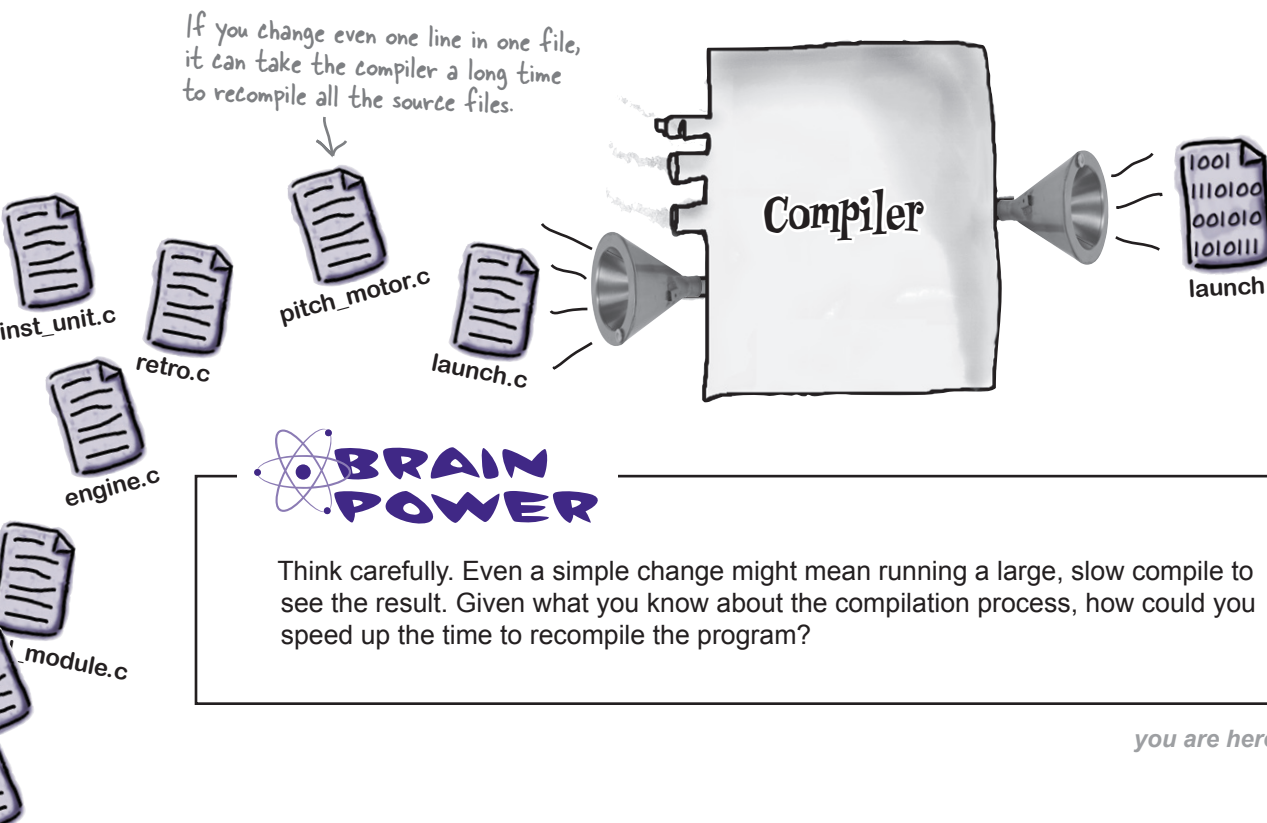


It's not rocket science...or is it?

Breaking your program out into separate source files not only means that you can *share code* between different programs, but it also means you can start to create *really large* programs. Why? Well, because you can start to break your program down into smaller **self-contained** pieces of code. Rather than being forced to have one *huge* source file, you can have lots of *simpler* files that are easier to understand, maintain, and test.

So on the plus side, you can start to create really large programs. The downside? The downside is...you can start to create really large programs. C compilers are really efficient pieces of software. They take your software through some very complex transformations. They can modify your source, link hundreds of files together without blowing your memory, and even optimize the code you wrote, along the way. And even though they do all that, they still manage to run quickly.

But if you create programs that use more than a few files, the time it takes to compile the code starts to become important. Let's say it takes a minute to compile a large project. That might not sound like a lot of time, but it's more than long enough to break your train of thought. If you try out a change in a single line of code, you want to see the result of that change as quickly as possible. If you have to wait a full minute to see the result of every change, that will really start to slow you down.



Don't recompile every file

If you've just made a change to one or two of your source code files, it's a waste to recompile every source file for your program. Think what happens when you issue a command like this:

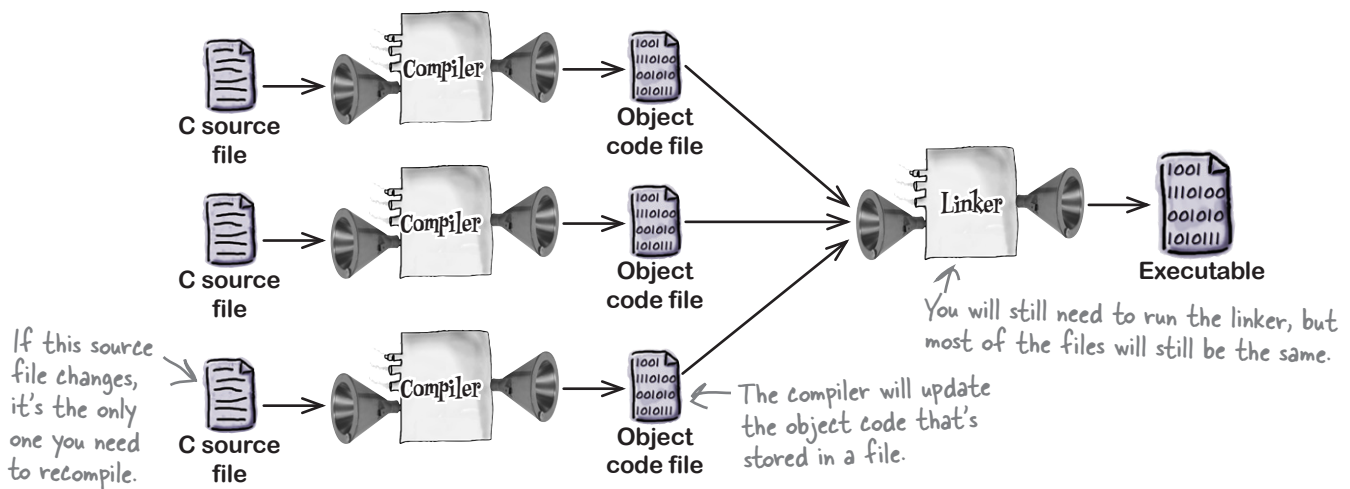
```
gcc reaction_control.c pitch_motor.c ... engine.c -o launch
```

Skipping a few filenames here.

What will the compiler do? It will run the preprocessor, compiler, and assembler for *each source code file*. Even the ones that haven't changed. And if the source code hasn't changed, the **object code** that's generated for that file won't change either. So if the compiler is generating the object code for every file, every time, what do you need to do?

Save copies of the compiled code

If you tell the compiler to save the object code it generates into a file, it shouldn't need to recreate it unless the source code changes. If a file *does* change, you can recreate the object code for that **one file** and then pass the whole set of object files to the compiler so they can be linked.



If you change a single file, you will have to recreate the object code file from it, but you *won't* need to create the object code for any other file. Then you can pass all the object code files to the linker and create a new version of the program.

So how do you tell gcc to save the object code in a file? And how do you then get the compiler to link the object files together?

First, compile the source into object files

You want object code for each of the source files, and you can do that by typing this command:

This will create object code for every file. `gcc -c *.c` ← The operating system will replace `*.c` with all the C filenames.

The `*.c` will match every C file in the current directory, and the `-c` will tell the compiler that you want to create an object file for each source file, but you don't want to link them together into a full executable program.

Then, link them together

Now that you have a set of object files, you can link them together with a simple compile command. But instead of giving the compiler the names of the C source files, you tell it the names of the object files:

This is similar to the compile commands you've used before. `gcc *.o -o launch` ← Instead of C source files, list the object files.
This will match all the object files in the directory.

The compiler is smart enough to recognize the files as object files, rather than source files, so it will skip most of the compilation steps and just link them together into an executable program called `launch`.

OK, so now you have a compiled program, just like before. But you also have a set of object files that are ready to be linked together if you need them again. So if you change just one of the files, you'll only need to recompile that single file and then relink the program:

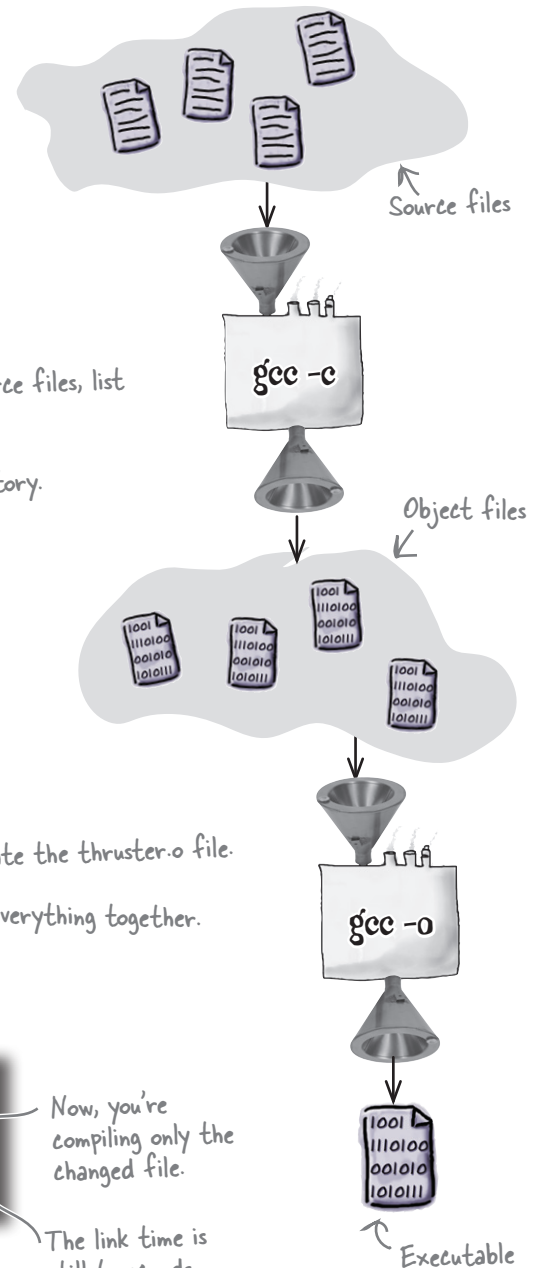
This is the only file that's changed. `gcc -c thruster.c` ← This will recreate the `thruster.o` file.
`gcc *.o -o launch` ← This will link everything together.

Even though you have to type two commands, you're saving a lot of time:

	Before	After	
Compile time:	2 mins 30 secs	2 secs	Now, you're compiling only the changed file.
Link time:	6 secs	6 secs	The link time is still 6 seconds.

Before, you were compiling every file. The build is 95% faster.

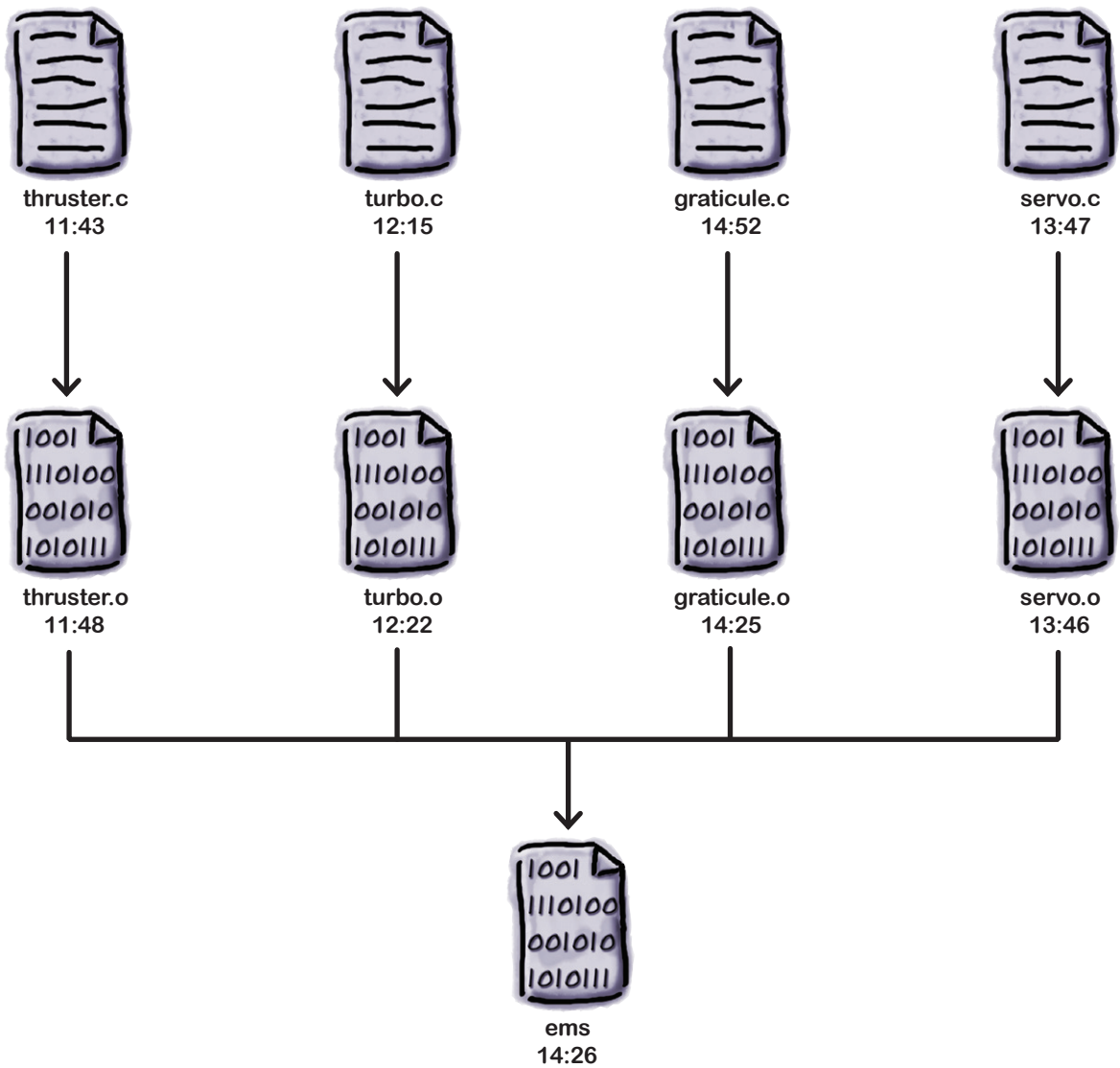
gcc -c will compile the code but won't link it.



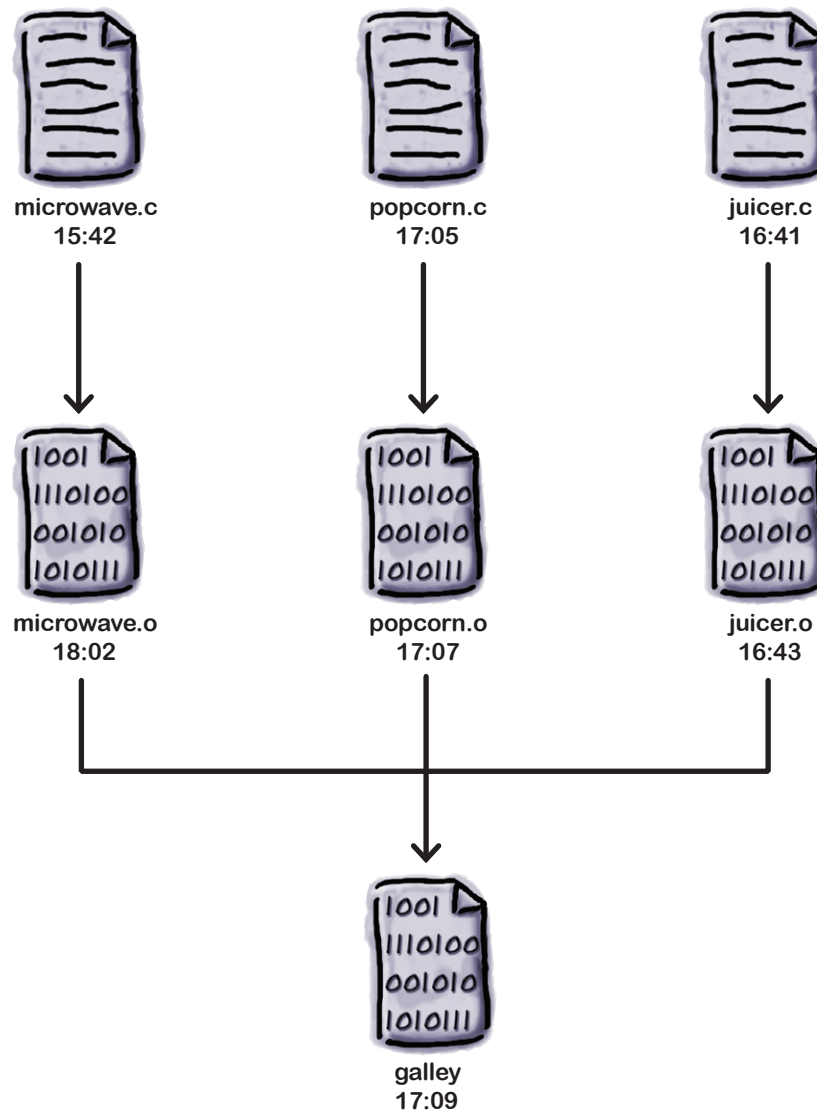


Long Exercise

Here is some of the code that's used to control the engine management system on the craft. There's a timestamp on each file. Which files do you think need to be recreated to make the `ems` executable up to date? Circle the files you think need to be updated.



And in the galley, they need to check that their code's up to date as well. Look at the times against the files. Which of these files need to be updated?



It's hard to keep track of the files



I **thought** the whole point of saving time was so I didn't have to get distracted. Now the compile is faster, but I have to think a **lot harder** about how to compile my code. Where's the sense in that?

It's true: partial compiles are faster, but you have to think more carefully to make sure you recompile everything you need.

If you are working on just one source file, things will be pretty simple. But if you've changed a few files, it's pretty easy to forget to recompile some of them. That means the newly compiled program won't pick up all the changes you made. Now, of course, when you come to *ship* the final program, you can always make sure you can do a full recompile of *every* file, but you don't want to do that while you're still developing the code.

Even though it's a fairly **mechanical process** to look for files that need to be compiled, if you do it manually, it will be pretty easy to miss some changes.

Is there something we can use to **automate the process**?



Automate your builds with the make tool

You can compile your applications really quickly in `gcc`, as long as you keep track of which files have changed. That's a tricky thing to do, but it's also pretty straightforward to automate. Imagine you have a file that is generated from some other file. Let's say it's an object file that is compiled from a source file:

If the `thruster.c` file is newer, you need to recompile.

`thruster.c` → `thruster.o`

If the `thruster.o` file is newer, you don't need to recompile.

How do you tell if the `thruster.o` file needs to be recompiled? You just look at the timestamps of the two files. If the `thruster.o` file is older than the `thruster.c` file, then the `thruster.o` file needs to be recreated. Otherwise, it's up to date.

That's a pretty simple rule. And if you have a simple rule for something, then don't think about it—**automate it**...

make is a tool that can run the compile command for you. The **make** tool will check the timestamps of the source files and the generated files, and then it will only recompile the files if things have gotten out of date.

But before you can do all these things, you need to tell **make** about your source code. It needs to know the details of which files depend on which files. And it also needs to be told exactly how you want to build the code.

What does make need to know?

Every file that **make** compiles is called a **target**. Strictly speaking, **make** isn't limited to compiling files. A target is any file that is *generated* from some other files. So a target might be a zip archive that is generated from the set of files that need to be compressed.

For every target, **make** needs to be told *two things*:



The dependencies.

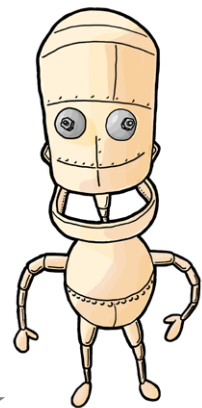
Which files the target is going to be generated from.



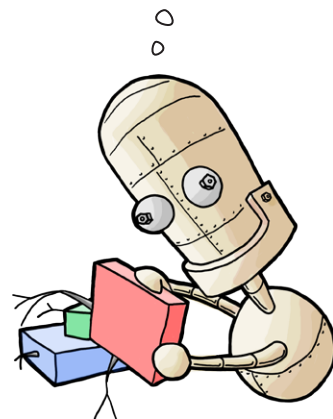
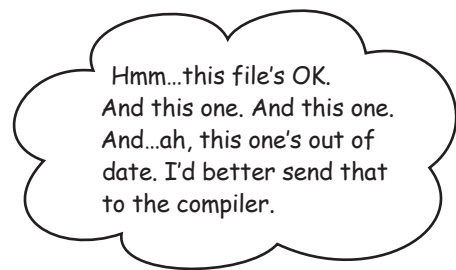
The recipe.

The set of instructions it needs to run to generate the file.

Together, the dependencies and the recipe form a **rule**. A rule tells **make** all it needs to know to create the target file.



This is **make**, your new best friend.



How make works

Let's say you want to compile *thruster.c* into some object code in *thruster.o*. What are the dependencies and what's the recipe?

`thruster.c` $\longrightarrow$ `thruster.o`

The *thruster.o* file is called the **target**, because it's the file you want to generate. *thruster.c* is a dependency, because it's a file the compiler will need in order to create *thruster.o*. And what will the recipe be? That's the compile command to convert *thruster.c* into *thruster.o*.

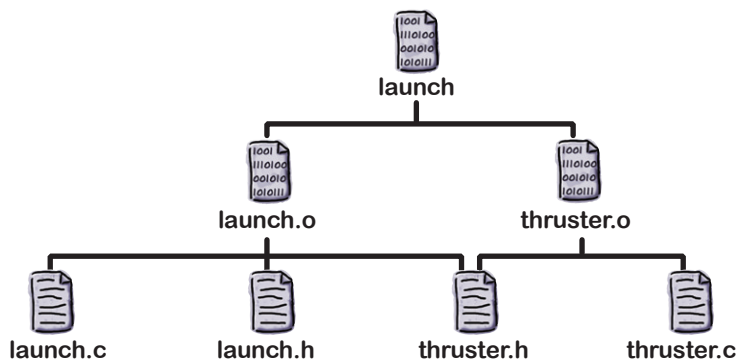
`gcc -c thruster.c` $\leftarrow$ This is the rule for creating *thruster.o*.

Make sense? If you tell the make tool about the dependencies and the recipe, you can leave it to make to decide when it needs to recompile *thruster.o*.

But you can go further than that. Once you build the *thruster.o* file, you're going to use it to create the *launch* program. That means the *launch* file can also be set up as a target, because it's a file you want to generate. The dependency files for *launch* are all of the *.o* object files. The recipe is this command:

`gcc *.o -o launch`

Once make has been given the details of all of the dependencies and rules, all you have to do is tell it to create the *launch* file. make will work out the details.



But how do you tell make about the dependencies and recipes? Let's find out.

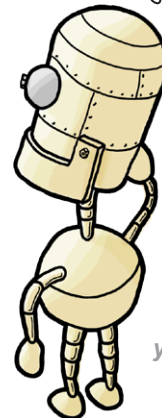


Watch it!

The make tool may have a different name on Windows.

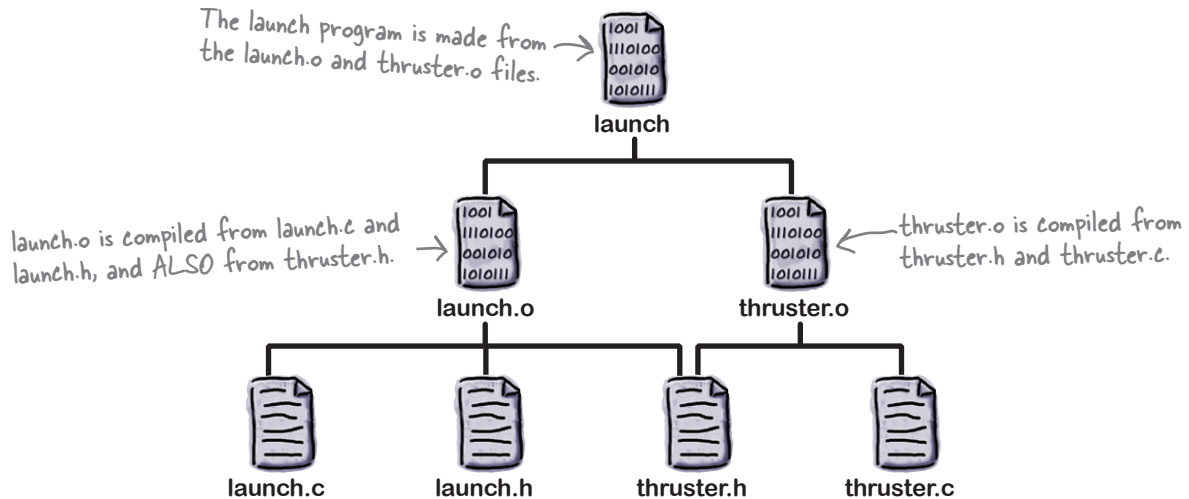
Because make came from the Unix world, there are different flavors of it available in Windows. MinGW includes a version of make called mingw32-make and Microsoft produce their own version called NMAKE.

So I've got to compile the launch program? Hmm... First I'll need to recompile *thruster.o*, because it's out of date; then I just need to relink *launch*.



Tell make about your code with a makefile

All of the details about the targets, dependencies, and recipes need to be stored in a file called either *makefile* or *Makefile*. To see how it works, imagine you have a pair of source files that together create the `launch` program:



The `launch` program is made by linking the `launch.o` and `thruster.o` files. Those files are compiled from their matching C and header files, but the `launch.o` file *also* depends on the `thruster.h` file because it contains code that will need to call a function in the `thruster` code.

This is how you'd describe that build in a makefile:

```

launch.o: launch.c launch.h thruster.h
    gcc -c launch.c
thruster.o: thruster.h thruster.c
    gcc -c thruster.c
launch: launch.o thruster.o
    gcc launch.o thruster.o -o launch
  
```

Handwritten annotations explain the makefile structure:

- "This is a target." points to `launch.o`.
- "A target is a file that is going to be generated." points to the `launch.o` line.
- "There are three RULES" points to the three rule blocks.
- "launch.o depends on these three files." points to the dependencies of `launch.o`.
- "This is a recipe for creating thruster.o." points to the recipe line for `thruster.o`.
- "The recipes MUST begin with a tab character." points to the tab character at the start of the recipe lines.



Watch it!

All of the recipe lines MUST begin with a tab character.

If you just try to indent the recipe lines with spaces, the build won't work.



— Test Drive —

Save your make rules into a text file called *Makefile* in the same directory; then, open up a console and type the following:

You are telling make to create the launch file.
 make first needs to create a launch.o with this line.
 make then needs to create thruster.o with this line.
 Finally, make links the object files to create the launch program.

```
File Edit Window Help MakeItSo
> make launch
gcc -c launch.c
gcc -c thruster.c
gcc launch.o thruster.o -o launch
```

You can see that make was able to work out the sequence of commands required to create the launch program. But what happens if you make a change to the *thruster.c* file and then run make again?

make no longer needs to compile launch.c.
 launch.o is already up to date.

```
File Edit Window Help MakeItSo
> make launch
gcc -c thruster.c
gcc launch.o thruster.o -o launch
```

make is able to skip creating a new version of *launch.o*. Instead, it just compiles *thruster.o* and then relinks the program.

there are no
Dumb Questions

Q: Is `make` just like `ant`?

A: It's probably better to say that build tools like `ant` and `rake` are like `make`. `make` was one of the earliest tools used to automatically build programs from source code.

Q: This seems like a lot of work just to compile source code. Is it really that useful?

A: Yes, `make` is amazingly useful. For small projects, `make` might not appear to save you that much time, but once you have more than a handful of files, compiling and linking code together can become very painful.

Q: If I write a makefile for a Windows machine, will it work on a Mac? Or a Linux machine?

A: Because makefiles calls commands in the underlying operating system, sometimes makefiles don't work on different operating systems.

Q: Can I use `make` for things other than compiling code?

A: Yes. `make` is most commonly used to compile code. But it can also be used as a command-line installer, or a source control tool. In fact, you can use `make` for almost any task that you can perform on the command line.



Tales from the Crypt

Why indent with tabs?

It's easy to indent recipes with spaces instead of tabs. So why does `make` insist on using tabs? This is a quote from `make`'s creator, Stuart Feldman:

"Why the tab in column 1? ... It worked, it stayed. And then a few weeks later I had a user population of about a dozen, most of them friends, and I didn't want to screw up my embedded base. The rest, sadly, is history."



Geek Bits

`make` takes away a lot of the pain of compiling files. But if you find that even it is not automatic enough, take a look at a tool called **autoconf**:

<http://www.gnu.org/software/autoconf/>

`autoconf` is used to generate makefiles. C programmers often create tools to automate the creation of software. An increasing number of them are available on the GNU website.



Make Magnets

Hey, baby, if you don't groove to the latest tunes, then you'll *love* the program the guys in the Head First Lounge just wrote! `oggswing` is a program that reads an Ogg Vorbis music file and creates a swing version. Sweet! See if you can complete the makefile that compiles `oggswing` and then uses it to convert a `.ogg` file:

This converts
`whitennerdy.ogg`
to `swing.ogg`.



`oggswing:`

.....

`swing.ogg:`

.....

`gcc oggswing.c -o oggswing`

`oggswing whitennerdy.ogg swing.ogg`

`whitennerdy.ogg`

`oggswing`

`[SPACES]`

`oggswing.h`

`oggswing.c`

`[TAB]`

`[SPACES]`

`[TAB]`